신희제 | Developer

포트폴리오 사이트 바로가기

+82 10-2390-2038 | heeheehee.hj@gmail.com | GitHub

소개

행사마다 2주씩 걸리던 등록 페이지를 공통화해 3~5일로 줄이고, JSP 레거시를 Next.js로 옮겨 성능 점수를 45에서 99로 올린 프론트엔드 개발자입니다.

- **시기를 보고 제안합니다**: 지금 고치기 어려운 결제 API는 일정 안에서 공통 로직으로 오류부터 막고 근본 수정은 고도화로 넘겼습니다. 새로 시작한 어드민은 시작 단계에서 모노레포와 UI 가이드를 제안해 첫 행사 일정에 맞춰 오픈했습니다.
- **근거를 정리해 함께 풉니다**: 결제 완료 처리를 서버로 옮기는 건 백엔드 팀에도 부담이 큰 변경이었습니다. 연결이 끊기는 케이스를 정리해 공유하며 함께 방법을 찾았고, 구조를 바꿔 결제 상태가 어긋나지 않게 했습니다.
- **결과가 있는 코드를 만듭니다**: 깔끔한 코드만큼 결과로 이어지는 일을 중요하게 여깁니다. 아무리 좋은 코드도 서비스에서 쓰이지 않으면 코드 조각에 불과하다고 생각해, 좋은 코드에만 매달리지 않고 지금 무엇이 먼저인지를 놓치지 않습니다. 아토믹 디자인을 적용했다가 정리를 위한 정리에 시간이 들고 개발 속도가 느려졌던 경험 이후로 더 그렇게 일합니다.

경력

이즈피엠피 (EZPMP) · ICT융합연구소 선임연구원

2022.05 - 2026.05 (4년) | 정규직 | 프론트엔드

- 전시회 주최사·지자체·키즈카페 고객사의 웹 서비스 프론트엔드 개발·운영 (지자체 관광 플랫폼 회원 1만+, 행사 플랫폼 연 2만+ 이용자)
- React·Next.js·TypeScript·TanStack Query·Zustand·Tailwind CSS 기반 신규 구축, JSP·jQuery 레거시 운영 및 Next.js 마이그레이션
- 주요 성과: Lighthouse 성능 45 → 99, 등록 페이지 개발 2주 → 3~5일, 결제 완료 처리 서버 이전 제안·반영, 어드민 첫 행사 일정 내 오픈

기술 스택

Frontend: React, Next.js, TypeScript, JavaScript, TanStack Query, Zustand, Tailwind CSS, PWA, Web Push

Backend: NestJS, Prisma, PostgreSQL, REST API, Swagger

테스트·빌드·배포: Vitest, Turborepo, pnpm workspace, Docker, GitLab CI/CD, ArgoCD, Vercel

AI·도구: Claude Code, Claude Design, Google Gemini API, Figma, Notion, Slack, IntelliJ IDEA

프로젝트

1. 어드민시스템 (전시 부스 신청 관리)

2026.02 - 2026.04 (3개월) | 프론트엔드 단독 | 팀 구성: 기획 2, 백엔드 1, 프론트엔드 1 실무

여러 전시회를 통합 운영하는 슈퍼어드민과, 참가업체 부스 신청부터 입금 확인·인보이스 발행까지 관리하는 프로젝트 어드민으로 구성된 2단계 어드민

어드민: ipsolution-admin.ezpmp.com | **Portfolio:** 상세보기

- **모노레포 구조 제안·설계**: 사용자 사이트와 어드민이 같은 신청 폼과 데이터 타입을 다뤄, 따로 만들면 같은 코드를 두 번 관리해야 했습니다. 두 앱을 하나로 묶는 모노레포(Turborepo + pnpm) 구조를 직접 제안하고 설계한 뒤, 컴포넌트·타입을 공유하며 개발했습니다. → 짧은 일정 안에 우선순위를 정해 **첫 행사 일정에 맞춰 오픈**

- **DynamicForm**: 전시회마다 신청 항목이 달라 매번 프론트 수정이 필요한 구조였습니다. 폼을 백엔드 설정에 따라 생성되게 바꿔 → **전시회가 추가돼도 프론트 수정 없이 설정만으로 대응**합니다.
- **상태 설계**: 서버 데이터는 TanStack Query로, 로그인 권한·선택한 프로젝트 같은 화면 상태만 Zustand에 두어 역할을 나눴습니다. 같은 데이터를 화면마다 따로 요청하지 않도록 도메인별 쿼리 훅으로 묶었습니다.
- **검증 규격화**: API 응답 타입을 먼저 정의하고 Zod로 런타임에서도 검증했습니다. AI가 만든 코드는 타입 체크와 **Vitest 테스트 216개**를 통과해야 반영되도록 했고, 결과는 CSV 리포트로 자동 정리했습니다. 실행 가이드는 README에 정리해 누구나 README만 보고 바로 테스트를 실행할 수 있게 남겼습니다.
- **UI 가이드 선제안**: 디자이너 없이 시작해 화면 기준이 없었습니다. 맡은 일은 아니었지만 가이드 페이지를 먼저 만들어 사업부서와 화면 기준을 합의했고, 이후 화면은 이 기준으로 개발했습니다.
- **배포 흐름**: QA 팀 없이 배포 전 확인이 필요해 테스트·운영 환경을 나눴습니다. 사업부서가 테스트 서버에서 확인한 뒤 운영에 반영하는 흐름을 만들었습니다 (GitLab CI/CD, ArgoCD Blue-Green).

기술 스택: Next.js, TypeScript, TanStack Query, Zustand, Tailwind CSS, Zod, Vitest, Turborepo

---

2. 스마트관광 플랫폼 (인제 / 용인)

2025.06 - 2025.11 (6개월) | 프론트엔드 | 팀 구성: 기획 2, 백엔드 2, 프론트엔드 3 실무

숙박·체험 예약, 투어패스 판매, 커뮤니티를 갖춘 지자체 관광 플랫폼 (4개 언어, 회원 1만+)

인제: tour.inje.go.kr | **용인:** itour.yongin.go.kr | **Portfolio:** 상세보기

- **고질적인 결제 이슈부터 바로잡기**: 결제 완료 처리를 프론트엔드가 맡고 있어, 결제 직후 연결이 끊기면 결제 상태가 어긋나는 문제가 반복되고 있었습니다. 프로젝트 초기에 이 문제부터 바로잡기로 하고, 프론트 방어 코드로 덮기보다 구조를 고치는 쪽을 택했습니다. 백엔드 팀에도 부담이 큰 변경이라 끊기는 케이스를 정리해 공유하며 함께 방법을 찾았고, 완료 처리를 서버로 옮겨 → **연결이 끊겨도 결제 상태가 어긋나지 않게** 됐습니다.
- **인증·결제 전담**: 가입과 결제는 이탈이 생기기 쉬운 구간이라 브라우저별 실패 케이스부터 정리했습니다. Safari ITP로 토큰이 유실되는 문제는 쿠키와 localStorage에 함께 저장하고 메모리 → 쿠키 → localStorage 순서로 복구하는 방식으로, 본인인증 팝업 차단은 리다이렉트로 바꿔 해결했습니다. → iOS에서도 로그인·결제가 정상 동작하게 됐고, Safari 확인을 배포 전 점검 항목에 넣었습니다. (JWT, NICE 본인인증, 카카오·네이버 로그인, KCP 결제, 4개 언어)

기술 스택: Next.js, TypeScript, Zustand, Tailwind CSS, KCP, Figma

---

3. O2MEET Next.js 마이그레이션

2024.12 - 2025.02, 2026.04 - 2026.05 | 프로젝트 구조·사용자 페이지(전시 안내·신청, 회원가입) 담당, 2차 우선순위 설계 주도 | 팀 구성: 프론트엔드 3 실무

JSP 기반 온라인 행사 플랫폼을 Next.js·TypeScript로 전환

Site: greenenertec.com | **Portfolio:** 상세보기

- **1차**: JSP 화면을 React로 옮기는 것 자체가 목표였고, 행사 일정으로 중단됐습니다.
- **2차**: 화면만 옮기기보다 느리고 무거운 원인을 먼저 걷어내야 한다고 보고, 세 가지를 우선했습니다. Next.js·TypeScript로 기술 스택을 바꾸고, 쓰지 않는 API 호출을 정리하고, 레거시 CSS를 Tailwind CSS로 전환했습니다. → **성능 점수 45 → 99, LCP 8.0초 → 0.8초, CLS 0.225 → 0** (Lighthouse 데스크톱, 같은 O2MEET 플랫폼의 JSP 사이트 대비 Next.js 사이트). 모바일 저속 환경의 LCP는 아직 느려, 메인 영상·이미지 최적화를 다음 과제로 남겼습니다.

기술 스택: Next.js, TypeScript, Zustand, Tailwind CSS, Docker, ArgoCD

---

4. O2MEET 온라인 행사 플랫폼 (운영)

2022.05 - 2026.05 | 상품·결제 프론트엔드 | 팀 구성: 기획 4, 백엔드 4, 프론트엔드 5 실무

- **상품·결제**: 연 2만+ 이용자의 온라인 전시관·컨퍼런스·티켓 구매 화면을 개발하고, KCP 결제와 쿠폰을 연동했습니다 (북키즈콘, SPP 등 행사 사이트).
- **등록 폼 공통화**: 행사마다 비슷하지만 조금씩 다른 등록 폼을 매번 새로 만들던 것을, 모든 행사에 공통인 백엔드 필수 항목 기준의 공통 함수로 분리했습니다. 행사별 안내 문구만 프로젝트에서 처리하도록 열어두고, 경우의 수를 늘리지 않는 것을 우선했습니다. → **신규 등록 페이지 개발 2주 → 3~5일**

- **결제 로직 공통화**: 결제 API가 범용으로 만들어져 쓰지 않는 값까지 필수였고, 새 페이지마다 결제 코드가 따로 있어 필수값 누락 오류가 반복됐습니다. API를 바꾸면 점검할 프로젝트가 너무 많고 행사 일정은 정해져 있어, 우선 필수값을 모두 채워 보내는 공통 결제 로직으로 → **새 페이지에서 반복되던 결제 오류를 막았습니다**. 근본 원인은 API 팀에 정리해 요청했고, 고도화 때 결제를 서비스로 분리하는 방향으로 잠정 결론을 냈습니다.

기술 스택: JSP, jQuery, KCP

---

5. POS/키오스크 (키즈카페 '디키디키')

2024.11 - 2025.02 | 키오스크 화면 프론트엔드 담당 | 팀 구성: 기획 1, 백엔드 2, 프론트엔드 3, C++ 1 | Portfolio 상세보기 실무

동대문역사문화공원역 어린이 키즈카페의 입장권·F&B 주문/결제용 키오스크와 POS. 웹 화면을 CefSharp 브라우저에 올려 영수증 프린터·서명패드·세컨드 모니터와 연동

- **단일 코드베이스·기기 통신**: 키오스크와 POS는 같은 결제 로직을 쓰지만 후처리(영수증·주방 주문서·서명패드)가 달라, 실행 환경만 분기하고 → **결제 로직은 한 벌로 유지**했습니다. CefSharp 메시지로 POS 프로그램과 연결해 영수증·입장권·주방 주문서 3종 출력과 프린터 상태를 연동했고, 연속 출력 시 프린터 버퍼가 충돌해 딜레이를 테스트하며 순차 출력으로 맞췄습니다.
- **결제 실패 복구·원격 잠금**: 사용자 취소와 시스템 오류를 응답 코드로 나눠, 취소는 재결제와 중복결제 방지로, 오류는 안내 후 30초 자동 복귀로 처리해 다음 고객이 기다리지 않게 했습니다. 매장 실무진과 이야기하며 인원이 넘치거나 다른 사정으로 키오스크를 받으면 안 되는 순간이 있다는 걸 알고, POS에서 키오스크를 원격으로 잠그는 기능을 만들었습니다.
- **기기 없이 개발, 현장에서 확인**: 개발/POS 환경을 분기해 실제 기기 없이 로컬에서 결제 흐름을 테스트했고, 현장 기기에서는 로그를 볼 수 없어 C++ 개발자와 협의해 화면을 다섯 번 누르면 오류 로그가 보이는 디버깅 모드를 만들었습니다.

기술 스택: JavaScript, jQuery, JSP, CefSharp, KCP

---

6. 중구(중고차 구매 가이드) (진행 중)

2026.10 - | 1인 · 2026.10.27 배포 | car-check-fawn-zeta.vercel.app | github.com/HeeJeShin/car-check 사이드

- 차종·연식을 넣으면 중고차를 살 때 확인할 고질병과 매장 점검 포인트를 알려주는 서비스입니다 (NestJS·Prisma·PostgreSQL + Next.js, 세대 98개). 돈이 걸린 사실 정보는 사람이 검증한 DB에서만 가져오고, 안내·추천만 Gemini에 맡겼습니다. 출처 URL이 없으면 '검증됨'으로 저장되지 않게 서버 규칙으로 막았습니다.
- **권한·심사 흐름**: 구글 로그인과 ADMIN·EDITOR 역할을 나누고, 고질병을 초안 → 검토 → 승인/반려로 심사합니다. 허용된 상태 전이만 서버에서 받고, 상태 변경과 변경 이력 저장을 한 트랜잭션으로 묶었으며, 두 사람이 동시에 승인하면 조건부 업데이트로 409를 돌려줍니다.
- **테스트·배포**: Playwright E2E와 GitHub Actions CI를 붙이고, CI를 통과해야만 배포되게 했습니다. Vercel(ISR) + Render + Neon으로 배포하고 Sentry로 에러를 수집합니다. AI는 키가 없어도 핵심 기능이 동작하도록 끔·모의·실제 3가지 모드로 두고, 같은 질문 캐시와 요청 제한을 넣었습니다.

---

7. HomeCare - 홈펌프 항암제 자가관리 웹앱

2026.05 - 2026.10 | 임상간호연구 개발 참여 · 1인 개발 (연구자료 제공 1인) | home-care-blond.vercel.app 사이드

- 현장의 어려움을 빠르게 덜어주는 것이 목적이라 서버 없이 바로 쓰는 웹앱(PWA)으로 만들었습니다.
- 노인 환자가 바로 판단할 수 있도록 수치 대신 그림과 "병원에 가야 해요" 같은 문장으로 설계했고, 의료진 요청은 응급 연락처와 PDF 리포트로 반영했습니다.
- Safari·Chrome·PWA 환경별로 웹 푸시 알림 권한을 처리했고, Claude Design으로 기획서를 만들어 의료진과 방향을 맞췄습니다.
- **결과**: 개발 문서(기획서·기술명세서)와 결과물을 연구팀에 전달하고 마무리했습니다.

---

8. 4J - 행사장 수용인원 자동계산

2026.01 | 사내 해커톤 · 1박 2일 사이드

- 1박 2일 팀 해커톤(기획 1, 프론트 3). Gemini는 도면에서 면적·출입구만 읽고, 수용인원은 기준에 따라 코드가 계산하도록 역할을 나눴습니다. · GitHub

학력

서울사이버대

2025.03 - 재학 중 (2027.02 졸업예정) | 컴퓨터공학과 (편입 3학년)

경민대학

2010.03 - 2012.02 졸업 | 호텔조리학과